

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Industry Problem Solving Task (Medium)

Assessment Tool Weightage: 35%

Faculty Name: Dr.G.Ayyappan

Student Reg. No: 192571049

Name: Hemalakshmi H

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5

6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I Hemalakshmi H - 192571049 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of	25%	Applies advanced and relevant	Applies appropriate concepts	Limited application of concepts	Incorrect or no

Engineering Concepts		concepts effectively to solve the problem	with minor errors		application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. File organization for 10 million product records (E-commerce)

Recommended: Clustered Sorted File + B+ Tree Index

Justification:

- 10 million records require efficient range queries (price, category, brand).
- Sorted file supports **sequential scans** with minimal I/O.
- B+ Tree index gives **$O(\log N)$** search and excellent range performance.
- Insert cost is moderate due to occasional page splits, but acceptable for product catalogs.
- Storage cost:
 - Record size ~ 200 bytes $\rightarrow 10M \times 200B = 2GB$
 - Block size 4KB $\rightarrow \sim 500,000$ blocks
 - B+ Tree height $\approx 3 \rightarrow$ only 3 I/Os per lookup.

Conclusion: Sorted file + B+ Tree provides the best balance of search speed, range queries, and storage efficiency.

2. Banking system indexing (account number + date range)

Indexing Strategy:

- **Primary index:** Composite B+ Tree on (account_no, date)
- **Secondary index:** B+ Tree on account_no alone
- **File organization:** Sorted by (account_no, date)

Justification:

- Point queries (specific account + date) $\rightarrow O(\log N)$
- Range queries (date range for an account) $\rightarrow$ locate first record, then sequential scan
- B+ Tree supports high fan-out $\rightarrow$ fewer disk I/Os
- Composite key ensures locality of all transactions for an account.

Conclusion: Composite B+ Tree is ideal for both point and range queries.

3. Healthcare system: multi-level indexing (PatientID + doctor name)

Design:

- **Primary file:** Sorted by **PatientID**
- **Primary index:** Dense **B+ Tree on PatientID**
- **Secondary index:** B+ Tree on **doctor_name** → **list of PatientIDs**
- **Multi-level:** doctor_name index → PatientID index → record

Justification:

- PatientID queries are fast due to primary B+ Tree.
- Doctor-wise queries use secondary index to fetch all patients under a doctor.
- Multi-level indexing reduces search space and avoids scanning entire file.

Conclusion: Multi-level B+ Tree indexing supports both unique and non-unique attributes efficiently.

4. Logistics: B-Tree vs B+ Tree for dynamic insert/delete

Recommendation: B+ Tree

Reasoning:

- Frequent insert/delete → B+ Tree handles leaf splits/merges efficiently.
- All records stored only in leaves → internal nodes remain stable.
- Leaf-level linking supports fast range scans (shipment by date, route).
- B-Tree stores keys in internal nodes → more splits, more I/O.

Conclusion: B+ Tree is better for dynamic, high-volume transactional workloads.

5. HR system hashing (5000 employees)

Comparison:

- **Static Hashing:**

- Fixed table size
- Good for stable datasets
- Overflow chains grow if distribution is skewed
- **Extendible Hashing:**
 - Directory expands/shrinks dynamically
 - Avoids long overflow chains
 - Better for unpredictable growth

Recommendation: Extendible Hashing

- 5000 employees may grow; extendible hashing ensures uniform bucket load and $O(1)$ average search.

6. Airline reservation: point (flight_no) + range (date)

Combined Index Structure:

- **Primary file:** Sorted by **date**
- **Index 1:** Composite B+ Tree on (**date, flight_no**)
- **Index 2:** Secondary B+ Tree on **flight_no**

Justification:

- Point queries (flight_no) → secondary index
- Range queries (date interval) → composite index
- Composite index ensures minimal I/O for date-based scans.

Conclusion: Combination of composite and secondary B+ Trees supports both query types efficiently.

7. University student DB: department + CGPA range

Design:

- **File organization:** Sorted by **department**, then **CGPA**
- **Primary index:** B+ Tree on **department**

- **Secondary index:** Within each department block, B+ Tree on **CGPA**

Justification:

- Department queries → direct block access
- CGPA range queries → sequential scan inside department block
- Reduces search space drastically compared to full-file scan.

Conclusion: Clustered sorted file + two-level B+ Tree indexing gives fast department and CGPA range queries.

8. Disk block utilization (Heap, Sorted, Hashed)

Given:

- Records = 100,000
- Record size = 50 bytes
- Block size = 4096 bytes

Records per block:

$$\left\lfloor \frac{4096}{50} \right\rfloor = 81$$

Blocks needed:

$$\left\lceil \frac{100000}{81} \right\rceil = 1235$$

Heap File:

- Utilization $\approx 100\%$
- No ordering
- Search cost $O(N)$

Sorted File:

- Same utilization
- Supports binary search → $O(\log N)$

Hashed File:

- Slightly more blocks due to overflow buckets

- Best for point queries $\rightarrow O(1)$ average
- Worst for range queries

Conclusion: Sorted file gives best balance; hashed best for point queries; heap simplest but slowest.

9. Social media: indexing 500M posts (user_id, timestamp, hashtag)

Composite Index Strategy:

- **Primary file:** Log-structured (append-only)
- **Index 1:** Composite B+ Tree on (user_id, timestamp)
- **Index 2:** Inverted index on hashtag $\rightarrow$ list of post_ids
- **Index 3:** Optional time-partitioned indexes (daily/weekly)

Justification:

- User timeline queries $\rightarrow$ composite index
- Hashtag searches $\rightarrow$ inverted index
- Timestamp ordering $\rightarrow$ natural in log-structured storage
- Scales to hundreds of millions of posts.

10. Performance comparison: Heap vs Sorted vs Hashed (1M transactions)

File Type Insert Time Delete Time Search Time (bill_id)

Heap	$O(1)$ append	$O(N)$	$O(N)$
Sorted	$O(N)$ (shift)	$O(N)$	$O(\log N)$
Hashed	$O(1)$ average	$O(1)$ average	$O(1)$ average

Analysis:

- Billing systems need fast lookup of transaction IDs $\rightarrow$ hashed file is ideal.
- Sorted file only useful for range queries (date-wise reports).
- Heap file is worst for search and delete.

Conclusion: Hashed file is best for supermarket billing with 1M daily transactions